Website Contact Form

A practical guide to the Formspree connection in the Liberty Congregation website

What is connected

The website is a static HTML, CSS, and vanilla JavaScript site hosted on Vercel. It does not have its own server or database. The contact form posts its fields from the visitor's browser to Formspree, which validates and routes accepted submissions to the notification email configured for this Formspree form.

Part	Purpose
index.html	Defines the form, field names, and Formspree endpoint.
script.js	Sends the form without leaving the page, handles success and failure, and opens an email-app fallback link when needed.
Formspree dashboard	Owns the form ID and notification-recipient configuration.
Vercel	Publishes the website files; it does not itself deliver form email.

Form endpoint: https://formspree.io/f/mbggelpw
Form ID: mbggelpw
The form ID is a public identifier used by the browser. It is not an email password or private API key.

How to create the endpoint next time

1. Create a form in Formspree and set its notification email in the dashboard.
2. Copy the endpoint. It has the shape https://formspree.io/f/FORM_ID.
3. Put that endpoint in the HTML action and JavaScript fetch URL. Keep both values identical.
4. Publish the site, send a test from the deployed URL, and check the response and inbox (including spam).

The email address shown on the website and the Formspree notification recipient are separate settings. Change the Formspree recipient in its dashboard; change visible mailto links in the website only if you want the public contact address to change too.

The code

HTML form (index.html)

Every field that should arrive by email needs a name. The action is the Formspree endpoint; method POST also provides a basic no-JavaScript fallback.

```
<form id="contact-form" method="post"
  action="https://formspree.io/f/mbggelpw">
  <label for="full-name">Full name</label>
  <input id="full-name" name="name" required>
  <label for="email">Email address</label>
  <input id="email" name="email" type="email" required>
  <label for="fellowship">Fellowship or Group</label>
  <input id="fellowship" name="fellowship">
  <label for="subject">Subject</label>
  <input id="subject" name="subject" required>
  <label for="message">Message</label>
  <textarea id="message" name="message" required></textarea>
  <p id="form-feedback" role="status" aria-live="polite"></p>
  <button type="submit">Send message</button>
</form>
```

Vanilla JavaScript / AJAX (script.js)

The browser sends FormData and requests a JSON response. The response status and any Formspree errors determine whether the success alert or fallback appears.

```
const endpoint = 'https://formspree.io/f/mbggelpw';
const data = new FormData(form);
data.set('_subject', `Website message: ${data.get('subject')}`);
const response = await fetch(endpoint, {
  method: 'POST',
  headers: { Accept: 'application/json' },
  body: data
});
const result = await response.json().catch(() => ({}));
if (!response.ok || result.errors?.length) {
  throw new Error('Formspree did not accept the submission.');
}
form.reset();
feedback.textContent = 'Your message has been submitted.';
window.alert('Your message has been submitted.');
```

FormData creates its own multipart boundary. Do not set Content-Type to application/json with a FormData body. The live project also disables the send button while sending, includes sender details in the fallback email draft, and restores the button afterward.

Status, troubleshooting, and repeatable checks

Result	Website behavior
Formspree accepts the submission	The form clears, a status message appears, and a success alert is shown. This confirms Formspree accepted it; check the inbox to confirm email delivery.
Formspree rejects or request fails	The form stays filled and shows an error with a mailto link. It opens the visitor's email app with message fields prefilled.
HTTP 522	A gateway timed out contacting the service. This usually points to an unavailable or slow provider server, not an invalid form field.

Next-time checklist

- Verify every input has a unique id, matching label, and useful name.
- Use type=email for email addresses and required for mandatory fields.
- Copy the correct Formspree endpoint into both HTML and JavaScript.
- Submit from the deployed HTTPS website, not just a local file.
- Check the response, Formspree submission history, inbox, and spam folder.
- Test a failed case to confirm the error and fallback are usable.
- Never put an email password or private API key in frontend JavaScript.

Current project files

index.html contains the form markup and endpoint. script.js handles submission, success/error states, and the mailto fallback. Formspree controls the notification recipient from its dashboard. For a new form, replace mbggelpw in both files and redeploy.

This guide documents the configured code. It does not include a live submission test.